TALLER FORMATIVO GUIADO: INFRAESTRUCTURA COMO CÓDIGO (IaC) CON TERRAFORM

Aprovisionamiento Declarativo On-Premise y Emulación Cloud en Salas 205M y 206M

Curso: Ingeniería de Software V (`09776 - TIC` / NRC: 11005) **Profesor:** Alejandro Muñoz Bravo

Espacio Académico: Laboratorios de Cómputo 205M / 206M (Red LAN / ZeroTier) **Fecha de Ejecución:** Semestre 2026-2 (Semana 13)

Modalidad: Individual o Parejas en Estaciones de Trabajo Linux (Usuario `docker`) **Ponderación Formativa:** 100 Puntos (Escala 0.0 a 5.0)

Nombre del Estudiante 1:	**Código:**	**Equipo / PC:**
Nombre del Estudiante 2 (opcional):	**Código:**	**Sala:** 205M [ ] 206M [ ]

1 CONTEXTO PEDAGÓGICO Y ARQUITECTURA DEL SISTEMA

En la sesión anterior (**Sesión 25: Orquestación con Kubernetes**), desplegamos pods, balanceadores y servicios distribuidos sobre un clúster multi-nodo. Sin embargo, surge una pregunta fundacional en la ingeniería de confiabilidad: *¿cómo aprovisionar de forma determinística, auditable y sin intervención manual el hardware virtualizado, las redes y el almacenamiento sobre los que corre el orquestador?*

El objetivo de esta práctica es dominar el aprovisionamiento declarativo mediante **HashiCorp Terraform**, implementando una arquitectura de microservicios de 3 capas en entorno **On-Premise** (utilizando el socket de Docker en las salas de cómputo 205M y 206M) y comprendiendo el flujo hacia entornos de virtualización empresarial (Proxmox VE) y nube pública (AWS).

Diagrama de la Pila de Cómputo del Laboratorio:

$$[\ \texttt{HCL Code}\ ] \longrightarrow [\ \texttt{Terraform Core + DAG}\ ] \Longleftrightarrow [\ \texttt{Provider Docker (gRPC)}\ ] \longrightarrow [\ \texttt{Docker Engine (Host 205M/206M)}\ ]$$

$$\downarrow$$

$$[\ \texttt{Bridge Network}\ ] \oplus [\ \texttt{Volume pgdata}\ ] \oplus [\ \texttt{PostgreSQL :5432}\ ] \oplus [\ \texttt{Redis :6379}\ ] \oplus [\ \texttt{Nginx :8080}\ ]$$

2 PROCEDIMIENTO EXPERIMENTAL Y TAREAS POR FASES

Fase 1: Preparación del Entorno Zero-Sudo en Salas 205M y 206M (15 min)

Los computadores de los laboratorios operan bajo el usuario de sesión `docker`. Dado que este usuario no posee privilegios de superusuario (`sudo`), la inicialización del entorno se realiza en el directorio local del usuario:

1. Clone el repositorio del curso o acceda a la carpeta del taller:

```
cd ~/work-space/cursos-content/IngesoftV/talleres/Taller_Formativo_Terraform_IaC_OnPremise_Cloud/codigo_base
```

2. Ejecute el script de verificación y preparación automática del laboratorio:

```
chmod +x setup_sala_205m_206m.sh && ./setup_sala_205m_206m.sh
export PATH="$HOME/.local/bin:$PATH"
```

3. Valide que Terraform esté listo y que el daemon de Docker responda correctamente:

```
terraform version && docker ps
```

Fase 2: Aprovisionamiento On-Premise de Stack de 3 Capas (30 min)

En la carpeta `01_docker_local_stack/`, inspecte la definición declarativa:

1. Inicialice el directorio y descargue el provider oficial de Docker:

```
cd 01_docker_local_stack
terraform init
```

2. Genere una simulación de los cambios y examine el Grafo de Dependencias:

```
terraform plan -out=tfplan
```

3. Aplique el plan compilado para materializar la infraestructura:

```
terraform apply tfplan
```

4. Valide que los recursos fueron creados correctamente con Docker y consulte las salidas:

```
terraform output
curl -I http://localhost:8080
docker ps --filter "name=ingesoft"
```

Fase 3: Gestión del Estado y Detección de Configuration Drift (25 min)

El archivo `terraform.tfstate` es la fuente única de verdad. En esta fase provocaremos un fallo por intervención manual no versionada (*ClickOps*):

1. Inspeccione los recursos registrados en el estado:

```
terraform state list
terraform state show docker_container.postgres_db
```

2. Simule un incidente manual en producción deteniendo y eliminando el contenedor de base de datos desde la consola de Docker:

```
docker stop ingesoft-lab-205m-postgres && docker rm ingesoft-lab-205m-postgres
```

3. Ejecute `terraform plan`. Observe cómo Terraform detecta la discrepancia entre el estado real y el deseado (*Drift Detection*) y propone recrear exactamente el recurso faltante sin alterar la red ni el frontend.
4. Reconcilie la infraestructura ejecutando `terraform apply -auto-approve` y verifique que el volumen de datos persistente se conservó intacto.

Fase 4: Análisis de Aprovisionamiento en Datacenter (Proxmox) y Cloud (AWS) (25 min)

Examine las carpetas `02_proxmox_k8s_node/` y `03_aws_emulated_localstack/`:

1. Identifique en el archivo `02_proxmox_k8s_node/main.tf` cómo la directiva `ipconfig0` y `clone` automatizan la inyección de Cloud-Init en un hipervisor KVM para crear los nodos del clúster de Kubernetes sin interacción manual.
2. En `03_aws_emulated_localstack/main.tf`, analice la equivalencia entre las redes bridge de Linux (`vmbr0`) y las nubes virtuales privadas (`aws_vpc`).

Fase 5: Modularización DRY y Shift-Left de Seguridad (25 min)

1. Formatee el código y valide la sintaxis estática:

```
terraform fmt -check && terraform validate
```

2. Cambie el valor de `enable_cache = false` en su archivo `terraform.tfvars` y aplique los cambios. Observe cómo la directiva condicional `count` retira el contenedor de Redis de forma limpia.
3. Al finalizar la práctica, destruya los recursos para liberar la memoria del computador de la sala:

```
terraform destroy -auto-approve
```

3 CUESTIONARIO DE ANÁLISIS CONCEPTUAL Y REFLEXIÓN CRÍTICA

Responda de manera sintética y justificada con su criterio de ingeniería:

1. **Configuration Drift:** Durante la Fase 3, ¿qué habría ocurrido si el contenedor eliminado tenía datos persistentes en su capa de escritura temporal en lugar de en el volumen `docker_volume`? ¿Cómo garantiza IaC la inmutabilidad de los datos?
2. **Seguridad del State:** Al abrir `terraform.tfstate`, observe el atributo `POSTGRES_PASSWORD`. ¿Por qué es una vulnerabilidad crítica versionar este archivo en un repositorio público de Git? ¿Qué alternativas empresariales existen para mitigar este riesgo?
3. **On-Premise vs. Cloud:** Compare el costo operativo y la latencia de desplegar 7 nodos de Kubernetes en Proxmox VE (Datacenter Icesi) frente a 7 instancias `t3.medium` en AWS. ¿En qué escenario empresarial recomendaría cada uno?

4 RÚBRICA DE EVALUACIÓN MODULARIZADA (ESCALA 0.0 A 5.0)

Criterio de Desempeño	Desempeño Esperado (Sobresaliente: 4.5 - 5.0)	Desempeño en Desarrollo (Aceptable: 3.0 - 4.4)	Peso
1. Preparación y Validación Zero-Sudo	El entorno en la sala 205M/206M se inicializa limpiamente con `setup_sala_205m_206m.sh`; Terraform y Docker se comunican sin privilegios de root.	Se requiere asistencia manual para configurar el PATH o los permisos del socket de Docker.	1.0 (20 %)
2. Aprovisionamiento On-Premise Declarativo	Stack de 3 capas materializado mediante `init`, `plan` y `apply`; red bridge, PostgreSQL, Redis y Nginx operativos y testeables vía curl.	El stack se despliega parcialmente o presenta colisiones de puertos en localhost.	1.5 (30 %)
3. Detección y Reconciliación de Drift	Demuestra detección exacta de discrepancia tras alteración manual con Docker CLI; reconcilia el stack preservando volúmenes persistentes.	Aplica cambios destructivos sin evidenciar la discrepancia en el plan.	1.0 (20 %)
4. Buenas Prácticas HCL y Parámetros	Código formateado con `fmt`; variables tipadas; outputs informativos y uso correcto de condicionales (`count`).	Código sin formatear, valores fijos *hardcodeados* o ausencia de outputs.	1.0 (20 %)
5. Análisis Crítico y Destrucción Limpia	Cuestionario conceptual respondido con profundidad técnica; destrucción limpia de recursos con `terraform destroy`.	Respuestas superficiales o recursos huérfanos dejados en la memoria del computador.	0.5 (10 %)

5 INSTRUCCIONES Y FORMATO DE ENTREGA

- **Entregable:** Repositorio Git con la carpeta del taller que contenga: `main.tf`, `variables.tf`, `outputs.tf`, archivo `terraform.tfvars` (sin secretos reales) y un informe breve en Markdown (`INFORME_LAB.md`) con las capturas de pantalla de la detección de Drift y las respuestas al Cuestionario de la Sección 3.
- **Canal institucional:** Plataforma de gestión de aprendizaje institucional (Intu / Brightspace Saman).
- **Criterio de Orden:** No incluir el archivo binario `terraform` ni la carpeta oculta `.terraform/` en el commit de entrega (utilizar un archivo `.gitignore` adecuado).